

Anatomy of a Kubernetes Cluster

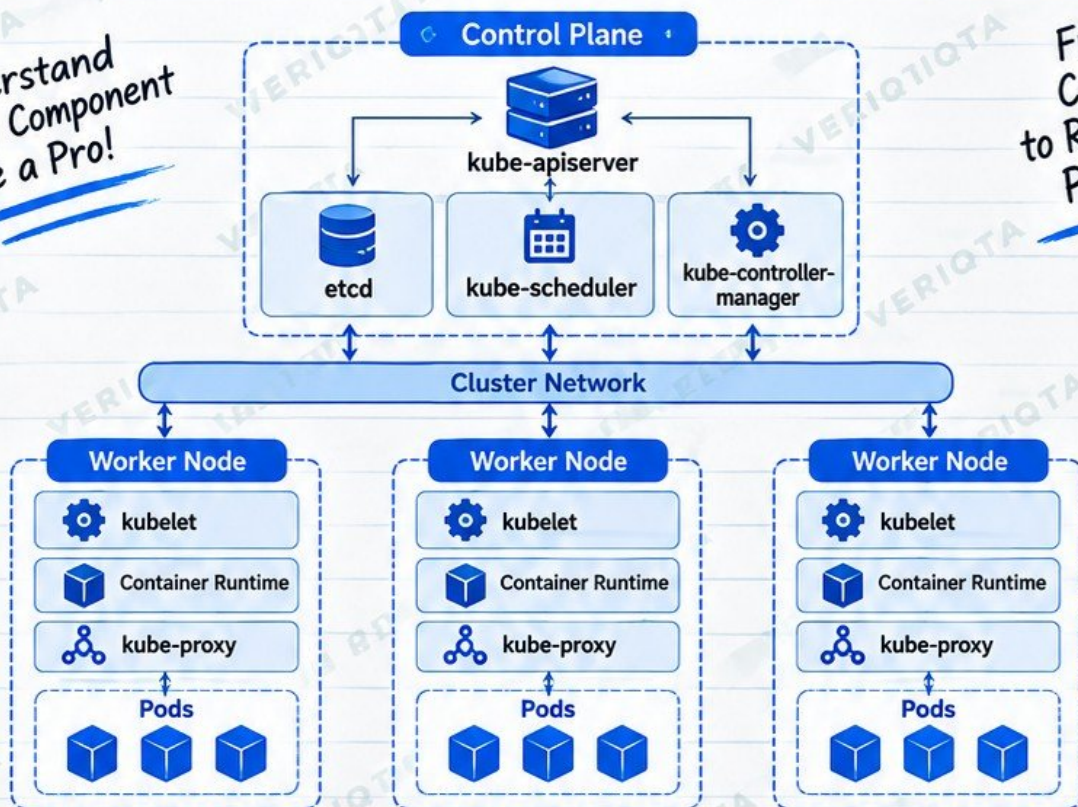


A COMPLETE 20-PAGE BEGINNER-TO-ADVANCED GUIDE

ARCHITECTURE | COMPONENTS | NETWORKING | STORAGE | SECURITY
OBSERVABILITY | HIGH AVAILABILITY | TROUBLESHOOTING | REAL-WORLD INSIGHTS

*Understand
Every Component
Like a Pro!*

*From
Concepts
to Real-World
Practice*




Clear
Explanations


Real
Examples


Practical
Diagrams


Production
Ready Knowledge


Build Real
Skills

VERIQTA

ENGINEER A BETTER TOMORROW

*More
Than Just
Theory*

Kubernetes Cluster

The Complete Mental Model

A Kubernetes cluster is a collection of components that work together to run, manage and scale containerized applications.

Key Concepts

- 1 What a Kubernetes cluster is
- 2 Control plane
- 3 Worker nodes
- 4 API-driven architecture
- 5 Desired state vs actual state
- 6 Controllers
- 7 Workloads
- 8 Cluster networking
- 9 Storage
- 10 Why multiple components

Desired vs Actual State



Desired State

- What you want to run
- Defined in YAML
- Stored in etcd

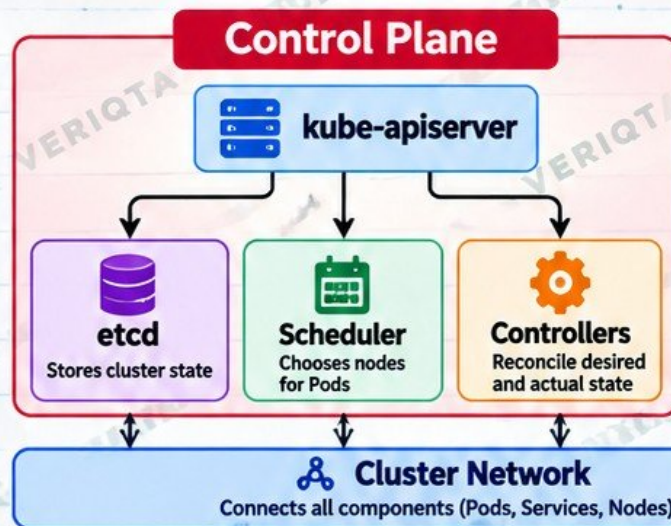
VS



Actual State

- What is currently running
- Continuously checked by controllers
- Differences are corrected automatically

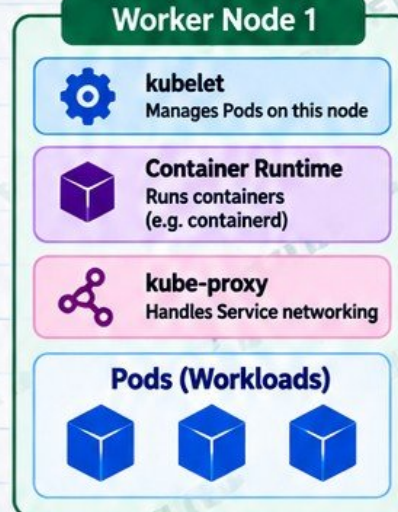
Control Plane



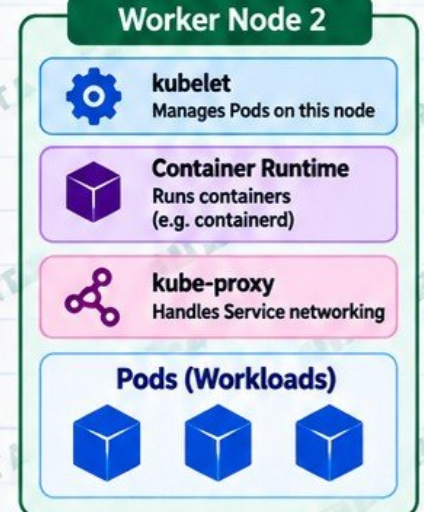
How It Works

- 1 You declare the desired state (e.g. a Deployment).
- 2 The API server validates and stores it in etcd.
- 3 Controllers watch for changes and take action.
- 4 The scheduler chooses a worker node.
- 5 Kubelet on the node runs the containers.
- 6 Networking and storage provide connectivity and persistent data.

Worker Node 1



Worker Node 2



Junior Engineer Takeaway

"Kubernetes is not one program. It is a collection of components continuously coordinating through the API."



Senior Connection

- Distributed control system
- Declarative state
- Reconciliation
- Eventual convergence



Junior Engineer Tip

Do not memorize components independently. Understand how information moves between them.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

ENGINEER A BETTER TOMORROW

Knowledge Builds Freedom

CONTROL PLANE ANATOMY

Inside the Control Plane

The control plane is the brain of the Kubernetes cluster.
It makes decisions, maintains the desired state and coordinates the entire cluster.

Key Components

- 1 kube-apiserver
- 2 etcd
- 3 kube-scheduler
- 4 kube-controller-manager
- 5 cloud-controller-manager

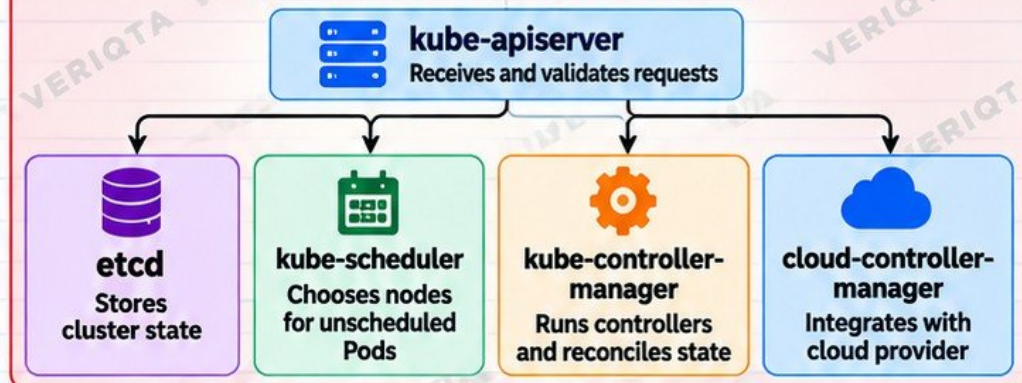
What is the Control Plane?

- A set of components that manage the cluster.
- Maintains the desired state.
- Schedules workloads.
- Runs controllers.
- Integrates with the cloud provider (if in a cloud).
- Communicates through the Kubernetes API.

Important Distinction

! The control plane does NOT run your application containers directly.
It decides and coordinates what should happen.
Your application containers run on worker nodes.

CONTROL PLANE



Component Responsibilities

Component	Main Responsibilities	What It Does NOT Do
kube-apiserver	- Receives and validates requests - Authentication & authorization - Persists data to etcd	- Does not run application containers - Does not schedule Pods
etcd	- Stores all cluster state (key-value) - Highly available and consistent	- Does not run workloads - Does not make scheduling decisions
kube-scheduler	Chooses the best node for unscheduled Pods	- Does not create Pods - Does not run containers
kube-controller-manager	- Runs controllers (Deployment, ReplicaSet, Node, Job, etc.) - Reconciles desired vs actual state	- Does not run application containers - Does not schedule Pods
cloud-controller-manager	- Integrates with cloud provider (VPC, ELB, disks, etc.) - Manages cloud-specific resources	- Does not run containers - Does not replace other controllers

How the Control Plane Works

- 1 You send a request (e.g. kubectl).
- 2 The API server validates and stores it.
- 3 Controllers watch for changes and act.
- 4 The scheduler assigns Pods to nodes.
- 5 The system continuously reconciles until the desired state is reached.

Senior-Level Concepts

- Static Pods (how control plane components run on nodes)
- Control-plane component placement
- Self-hosted control planes (e.g. kubeadm)
- Managed Kubernetes control planes (e.g. EKS, GKE, AKS)
- Control-plane failure domains and high availability

Junior Engineer Tip

Do not memorize components independently. Understand how information moves between them.

kube-apiserver

The Front Door of Kubernetes

The kube-apiserver is the central communication point of a Kubernetes cluster. It receives, validates and processes all requests to the cluster.

Who Communicates With It?

-  `kubectl` (you, the user)
-  kubelet (worker nodes)
-  Scheduler
-  Controllers
-  Operators
-  CI/CD systems
-  Admission controllers
-  External automation (scripts, tools, APIs)

Important Points

-  The API server is stateless.
-  It does not run your application containers.
-  It decides and coordinates what should happen.
-  All components communicate with it through the Kubernetes API.

Remember



The control plane does NOT run your application containers directly. It manages the desired state and coordinates the cluster.

Request Flow



Key Concepts

 REST API	The kube-apiserver exposes a RESTful API over HTTPS.
 Kubernetes resources	Everything in Kubernetes (Pods, Services, Deployments etc.) is a resource.
 API groups	Resources are organized into groups (e.g. "", apps, batch, rbac, etc.).
 Versions	Each resource has versions (e.g. v1, v1beta1).
 Resource endpoints	Access resources via endpoints like <code>/api/v1/pods</code> or <code>/apis/apps/v1/deployments</code> .
 Watch operations	Clients can "watch" resources to get real-time updates when objects change.

Example Command

```
kubectl get pods
```

-  Sends an HTTP GET request to the API server.
-  The API server authenticates and authorizes you.
-  It retrieves the Pod list (from etcd via its cache).
-  It returns the results to your terminal.

What really happens?

`kubectl` uses the Kubernetes REST API. The API server processes the request, checks permissions, retrieves the data and returns it in JSON (which `kubectl` formats).



Troubleshooting

Check if the API server is healthy:

```
kubectl get --raw=/readyz
```

- Returns the health status of the API server.
- Useful for checking if the API server is responding.



Senior-Level Topics

- API server horizontal scaling
- Watch streams and long-lived connections
- API Priority and Fairness (APF)
- Request throttling and rate limits
- API server saturation (high load)
- Admission latency and slow webhooks
- Caching and etcd read/write patterns
- High availability and load balancing



Junior Engineer Tip

Do not just memorize the components. Understand how requests flow through the API server and what happens at each step.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

ENGINEER A BETTER TOMORROW

Knowledge
Builds Freedom

etcd

The Cluster's Source of Truth

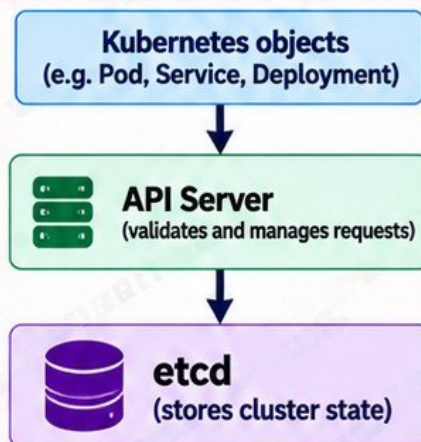
Reliable
Clusters
Start with
Reliable Data

etcd is a distributed key-value database that stores the entire Kubernetes cluster state. It is the single source of truth for the cluster.

What Does etcd Store?

-  Deployments
-  Pods
-  Secrets
-  ConfigMaps
-  Services
-  Nodes
-  RBAC objects
-  Metadata
-  Desired configuration

How It Fits in Kubernetes



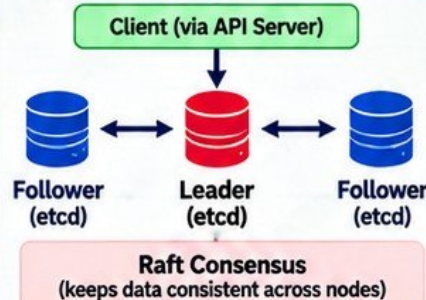
Key Points to Understand

- 1 etcd is a distributed key-value database.
- 2 It stores the desired and actual state of the cluster.
- 3 Pods do not ask etcd directly.
- 4 kubelet does not normally ask etcd directly.
- 5 All components interact with etcd through the kube-apiserver.
- 6 If etcd is unavailable, the cluster cannot be managed.
- 7 etcd does not run your application containers.

It Is a Distributed System

- Uses the Raft consensus algorithm for consistency.
- Requires a quorum (majority of members) to operate.
- Best practice: odd number of members (e.g. 3, 5, 7).
- Has a leader and follower nodes (leader handles writes).

Typical etcd Cluster (3 nodes)



Important Components

-  WAL (Write-Ahead Log)
Records all changes.
-  Snapshots
Point-in-time backup of data.
-  Compaction
Removes old, unnecessary data.
-  Defragmentation
Reclaims disk space.
-  Disk latency
High latency can affect performance.
-  Database size
Needs monitoring and regular maintenance.

Production Awareness

-  Loss of application Pods can be recoverable (they can be recreated).
- Loss of etcd without a backup can mean loss of the entire cluster state.
- Always take regular snapshots.
- Monitor disk usage, latency and cluster health.

Senior-Level Concepts

- Raft consensus algorithm
- Quorum and leader election
- WAL and data durability
- Snapshots and restore
- Compaction and defragmentation
- etcd performance tuning
- API server and etcd scaling
- Failure scenarios and recovery
- Managed vs self-hosted etcd (e.g. EKS, GKE, AKS)

Remember

- ✓ etcd is the heart of the cluster state.
- ✓ Keep it highly available.
- ✓ Use an odd number of nodes.
- ✓ Take regular backups.
- ✓ Monitor its health and size.
- ✓ If etcd is down, you cannot create, update or manage resources.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

ENGINEER A BETTER TOMORROW

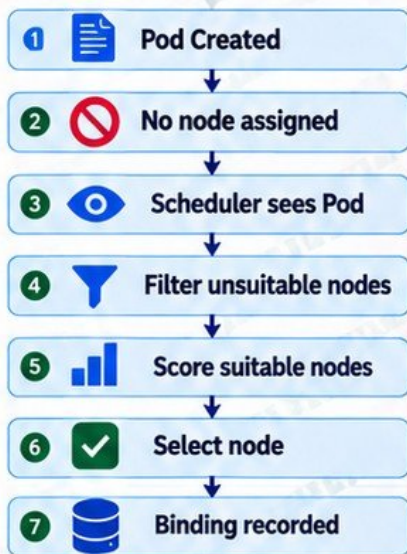
Knowledge
Builds Freedom

kube-scheduler

How Kubernetes Chooses a Node

The kube-scheduler watches for newly created Pods with no assigned node, finds the best node based on rules and available resources, and binds the Pod to that node.

Scheduling Flow



What the Scheduler Considers

-  CPU capacity and requests
-  Memory capacity and requests
-  Taints on nodes
-  Tolerations in Pod spec
-  Node selectors (specific nodes)
-  Node affinity (prefer or require nodes)
-  Pod anti-affinity (spread Pods)
-  Topology (zones, regions, racks)
-  Volume constraints (e.g. local storage)

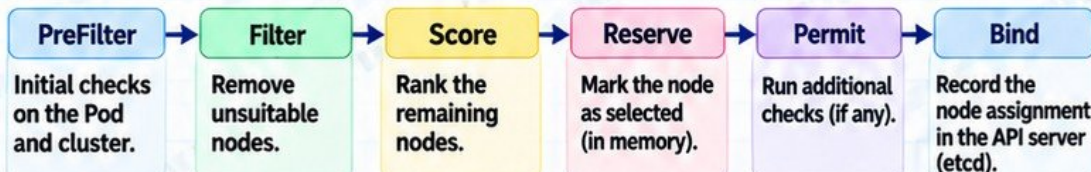
Resource Requests Example

resources:
requests:
cpu: "500m"
memory: "512Mi"

Why requests matter:

- The scheduler uses requests (not limits) to decide if a node has enough resources.
- Requests guarantee that the Pod will have the specified resources.
- If a node does not have at least the requested CPU and memory, the Pod will not be scheduled there.

Inside the Scheduling Process (Scheduling Framework)



Advanced Topics

- Scheduler profiles
- Custom schedulers
- Multiple schedulers
- Extender framework
- Plugin architecture

Common Failure

0/3 nodes are available:

- 3 Insufficient CPU
- 3 Insufficient memory
- 3 node(s) had taint {key: value}

This means the scheduler could not find any node that meets the Pod's requirements.

How to Investigate

- 1 Check the Pod events
`kubectl describe pod <pod-name>`
- 2 Check node resources
`kubectl get nodes -o wide`
- 3 Check taints on nodes
`kubectl describe node <node-name>`
- 4 Check resource requests in the Pod spec
`kubectl get pod <pod-name> -o yaml`

Key Takeaways

- ✓ The scheduler does not run Pods. It only decides which node to use.
- ✓ It uses multiple factors, not just CPU and memory.
- ✓ Requests are critical for scheduling.
- ✓ If no suitable node is found, the Pod remains in Pending state.



Junior Engineer Tip

- ✓ Do not just think "the scheduler places Pods". Understand the rules, resources and constraints it uses to make the decision.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

ENGINEER A BETTER TOMORROW

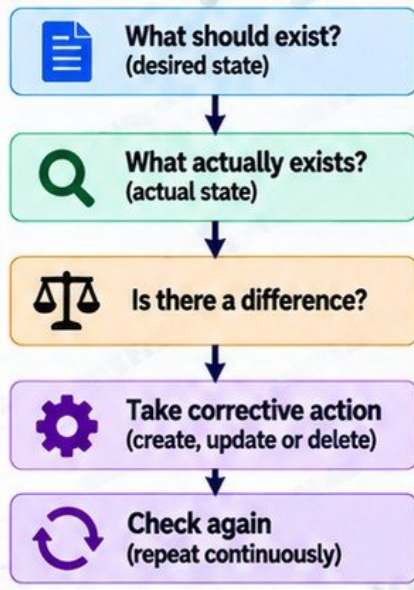
Knowledge
Builds Freedom

Controllers

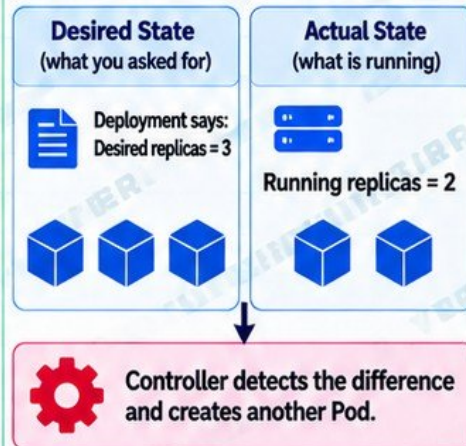
The Reconciliation Engine

Controllers continuously watch the cluster state and work to make the actual state match the desired state. This reconciliation loop is what makes Kubernetes Kubernetes.

How Reconciliation Works



Real Example: Deployment Controller



Common Controllers

- Deployment Controller**
Manages Deployments and ReplicaSets.
- ReplicaSet Controller**
Maintains the desired number of Pods.
- Node Controller**
Monitors node health and status.
- Job Controller**
Handles Jobs and CronJobs.
- EndpointSlice Controller**
Maintains EndpointSlices for Services.
- Namespace Controller**
Handles Namespace lifecycle.

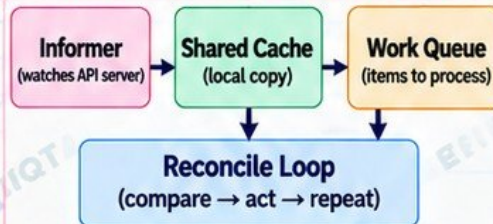
Senior-Level Concepts

- Informers and shared caches
- Work queues and rate limiting
- Idempotency (safe to run multiple times)
- Reconciliation loops
- Controller patterns
- Custom controllers and Operators
- Event-driven architecture
- Leader election
- Finalizers
- Status subresource

What Controllers Watch

- ✓ API server for resource changes
- ✓ Current state vs desired state
- ✓ Events and object updates
- ✓ Status of dependent resources

Inside a Controller (High-Level)



Key Lesson

- ✓ Kubernetes is driven by reconciliation, not by one-time commands.

Controller Actions

- ✓ Create missing resources
- ✓ Update incorrect resources
- ✓ Delete unwanted resources
- ✓ Retry until the desired state is reached
- ✓ Handle failures gracefully
- ✓ Continuously run (always watching)

Junior Engineer Tip

Don't just create objects and forget them. Understand which controller manages each resource and how it brings the cluster to the desired state.

WORKER NODE ANATOMY

Worker Node: Where Applications Actually Run

A worker node is a machine (physical or virtual) that runs your application workloads (Pods). It provides the compute, storage and networking resources needed for containers to run.

Components on a Worker Node

- kubelet**
Agent that communicates with the API server and manages Pods on this node.
- Container runtime**
Runs containers (e.g. containerd, CRI-O).
- kube-proxy or eBPF dataplane**
Handles Service networking (iptables, IPVS or eBPF).
- CNI components**
Provides Pod networking (e.g. Calico, Cilium, Flannel).
- CSI components**
Handles storage (mounts volumes from storage backend).
- Pods**
Your application containers.

Worker Node



Worker Node

Node IP: 10.0.1.10
OS: Linux
CPU, Memory, Disk, Network



kubelet



Container runtime (e.g. containerd)



kube-proxy / eBPF



CNI components (e.g. Calico, Cilium, Flannel)



CSI components (e.g. EBS, EFS, Ceph, NFS)

Pods (on this node)



Pod 1
Pod IP: 10.244.1.5



App container



Sidecar (optional)



Volumes



Pod 2
Pod IP: 10.244.1.6



App container



Volumes

What Runs Where?



On the Node (not inside a Pod)

- kubelet
- container runtime
- kube-proxy / eBPF
- CNI components
- CSI components
- system services
- node OS and filesystem
- CPU, memory, disk, network



Inside the Pod

- application containers
- sidecar containers
- init containers
- ephemeral containers
- mounted volumes
- your application code and dependencies

Node and Pod Networking

- Node IP: IP address of the worker node (e.g. 10.0.1.10).
- Pod IP: Each Pod gets its own IP from the cluster network (e.g. 10.244.1.5).
- Pods can communicate with each other across nodes (via CNI).
- Services (via kube-proxy) provide stable access to Pods.

Node Resource Consumption



CPU

Used by system processes, kubelet, containers.



Memory

Used by OS, kubelet, containers (requests/limits).



Disk / Filesystem

Stores OS, container images, logs, volumes.



Network

Used for Pod traffic, Service traffic, cluster traffic.

Advanced Concepts (Senior Level)



NodeAllocatable

Amount of resources available for Pods.



kube-reserved

Resources reserved for Kubernetes components.



system-reserved

Resources reserved for OS/system processes.



Eviction thresholds

When the node starts evicting Pods (e.g. low memory).



Capacity vs Allocatable

Total resources vs what is available for scheduling.

Key Takeaways



- Worker nodes run your application containers. System components on the node enable Pods to run.
- Each Pod has its own IP address.
- Node resources are shared between system components and Pods.
- Understand resource reservations and eviction to avoid node failures.



Junior Engineer Tip

A worker node is more than just a server. Understand what runs on the node vs inside the Pod. This helps you troubleshoot issues faster.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

ENGINEER A BETTER TOMORROW

Knowledge
Builds Freedom

kubelet

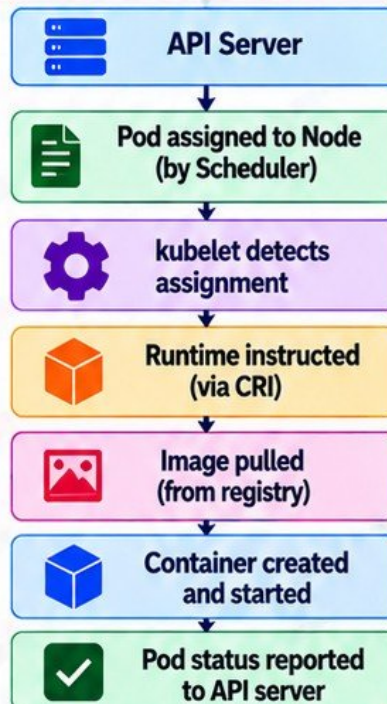
The Node Agent

kubelet is the agent that runs on every worker node. It makes sure the Pods assigned to the node are running, healthy, and reported back to the control plane.

What kubelet Does

- ✓ Registers the node with the API server.
- ✓ Watches for Pods assigned to the node.
- ✓ Talks to the container runtime (via CRI).
- ✓ Pulls container images.
- ✓ Creates and manages containers.
- ✓ Mounts volumes for Pods.
- ✓ Executes liveness, readiness and startup probes.
- ✓ Reports Pod and node status to the API server.
- ✓ Manages the full container lifecycle (start, stop, restart).

kubelet Workflow



Important Clarification



kubelet does NOT choose the node.

The scheduler chooses the node. kubelet only works on the node it is assigned to.

Key Interactions

-  **API Server**
(receives Pod specs, sends status)
-  **Container Runtime**
(containerd, CRI-O, etc.)
-  **Storage (CSI)**
(mounts volumes)
-  **CNI**
(works with network plugins)
-  **Health Checks**
(executes probes)

Senior-Level Concepts

-  **Pod Lifecycle Event Generator**
Generates events for Pod lifecycle changes.
-  **CRI (Container Runtime Interface)**
Standard interface to container runtimes.
-  **kubelet configuration**
Flags and config file (e.g. `/var/lib/kubelet/config.yaml`).
-  **NodeLease**
Sends heartbeats to indicate node health.
-  **Certificate rotation**
Automatically rotates kubelet client certificates.
-  **Eviction manager**
Monitors resource usage and evicts Pods when needed.

Troubleshooting

Check kubelet status:

```
systemctl status kubelet
```

View logs:

```
journalctl -u kubelet
```

Common checks:

- Is kubelet running?
- Are there errors in the logs?
- Is the node registered?
- Are Pods stuck in ContainerCreating?
- Are there image pull errors?



Key Takeaways

- ✓ kubelet is the node agent.
- ✓ It makes Pods happen on the node.
- ✓ It communicates with the API server and container runtime.
- ✓ It manages containers, volumes, networking and health checks.
- ✓ It reports the actual state of the node and Pods.



Junior Engineer Tip

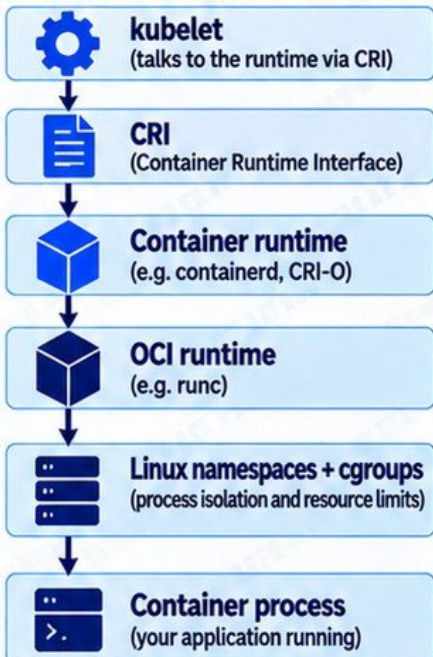
When a Pod is not running, always check the kubelet logs on the node. It tells you exactly what happened.

Container Runtime

How Pods Become Processes

Kubernetes does not itself execute containers. The container runtime does.
The runtime is responsible for creating and running containers on the node.

The Container Runtime Flow



What Is a Container Runtime?

A container runtime is software that pulls container images, creates containers, manages their lifecycle and runs them on the host operating system.

Common Runtimes in Kubernetes



containerd

- Default runtime in many distributions
- Lightweight and production ready
- Maintained by the CNCF



CRI-O

- Open source
- Designed specifically for Kubernetes
- Common in OpenShift

Other Important Components



runc

- OCI compliant low-level runtime
- Actually creates the container
- Works with containerd and CRI-O



crun

- Alternative OCI runtime
- Lightweight and fast
- Supports cgroups v2

Key Concepts



CRI (Container Runtime Interface)

- Standard interface between kubelet and the container runtime.
- Allows different runtimes (containerd, CRI-O) to be used.



OCI (Open Container Initiative)

- Defines the image format and runtime specification.
- Ensures containers are portable across runtimes.



Namespaces

- Provide isolation between containers (e.g. PID, network, mount, IPC, UTS).
- Make containers feel like separate environments.



cgroups (control groups)

- Limit and track resource usage (CPU, memory, I/O).
- Prevent one container from consuming all node resources.

The Docker Confusion



Docker Engine is NOT required by Kubernetes.

- Kubernetes used to support Docker via dockershim (removed in v1.24).
- Today, Kubernetes uses containerd or CRI-O via the CRI.
- You can still build container images with Docker, but Docker Engine is not used to run containers in Kubernetes.

Build with Docker if you want.
Run with containerd or CRI-O.

Advanced Topics (Senior Level)



containerd-shim

- Keeps containers running independently of containerd.
- Handles the container lifecycle after creation.



Runtime Classes

- Allow different runtimes for different Pods (e.g. runc, gVisor, Kata Containers).
- Useful for security or performance needs.



Sandbox Containers (Pod Sandbox)

- A lightweight infrastructure container that holds the Pod's network namespace.
- Created by the runtime before app containers.



Low-level Runtime Isolation

- Runtimes can provide stronger isolation (e.g. gVisor, Kata Containers, Firecracker).
- Useful for multi-tenant or untrusted workloads.

Real-World Example



On a node using containerd:

```
# Check runtime
crictl info
# List containers
crictl ps -a
# Check runtime version
containerd --version
```



Junior Engineer Tip

You don't need to memorize every runtime. Understand the layers and how kubelet uses the CRI to talk to the runtime. Focus on how containers are created, run and isolated.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

POD ANATOMY

Inside a Pod

A Pod is the smallest deployable unit in Kubernetes. It can contain one or more containers that share the same network, storage and lifecycle.

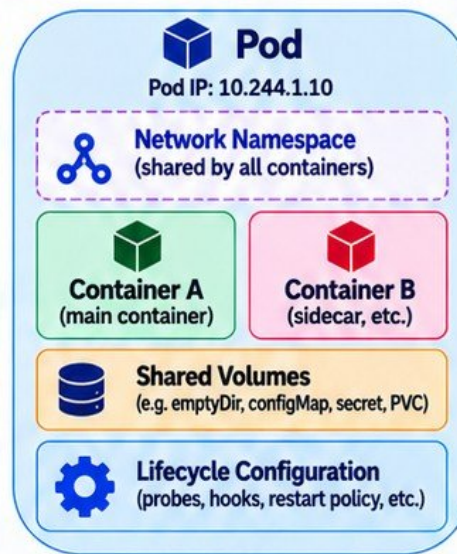
What Is a Pod?

- A Pod is a group of one or more containers that run on the same node.
- Containers in a Pod share network, storage and lifecycle.
- A Pod has a unique IP address.
- A Pod represents a running process in your cluster.
- Pod ≠ Container (A Pod can have multiple containers).

Pod ≠ Container

A container is inside a Pod.
A Pod can have one or more containers.

Anatomy of a Pod



Containers in the Same Pod Share

- ✓ Network namespace (same IP address)
- ✓ localhost (can communicate via localhost)
- ✓ Ports namespace (can use different ports)
- ✓ Volumes (when configured)
- ✓ Lifecycle (created and deleted together)

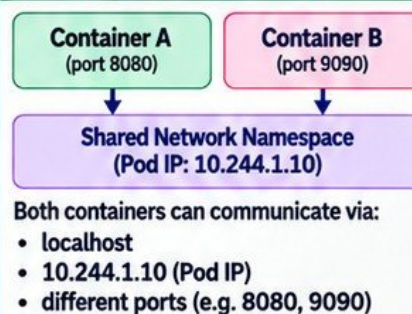
Each Pod Has

- Pod IP (unique IP address)
- Network namespace
- Shared volumes
- Containers
- Lifecycle configuration

Types of Containers in a Pod

- Main Container**
Runs the main application.
- Sidecar Container**
Runs a supporting process (e.g. logging, monitoring, proxy).
- Init Container**
Runs before the main containers start. Used for setup tasks.
- Ephemeral Container**
Used for debugging. Added to a running Pod.

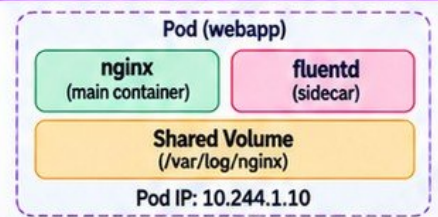
Pod Network Model



Senior-Level Concepts

- Pause / Sandbox Container**
A minimal container that holds the network namespace for the Pod.
- Pod Sandbox**
Created by the container runtime.
- Linux Namespace Sharing**
Network, IPC, UTS (hostname), etc.
- Lifecycle Hooks**
postStart and preStop hooks.
- Termination Sequence**
PreStop → SIGTERM → grace period → SIGKILL.

Real-World Example (Pod with Sidecar)



Key Takeaways

- ✓ A Pod can run one or more containers.
- ✓ Containers in a Pod share network and storage.
- ✓ A Pod has its own IP address.
- ✓ Pods are scheduled and managed as a single unit.
- ✓ Different container types have different purposes.
- ✓ Understand the Pod sandbox and namespaces for deeper troubleshooting.



Junior Engineer Tip

Think of a Pod as a small server for your application, where containers work together, share the same network and storage, and live and die together.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

From Deployment to Running Container

A complete end-to-end journey of how a Deployment becomes running containers in Kubernetes.
Multiple components work together, each watching and reacting to change.

The Flow (Step by Step)

- 1  **kubectl apply**
You create a Deployment using kubectl.
- 2  **API Server**
Receives, validates and stores the Deployment object.
- 3  **etcd**
Persists the Deployment object (desired state).
- 4  **Deployment Controller**
Watches for Deployments and creates a ReplicaSet.
- 5  **ReplicaSet**
Creates the desired number of Pod objects.
- 6  **Pod Objects**
Pods are created and stored in etcd (pending state).
- 7  **Scheduler**
Watches for unscheduled Pods and selects the best node.
- 8  **Node Selected**
Pod is bound to a node (nodeName is set).
- 9  **kubelet**
On the node, kubelet sees the assigned Pod and starts working.
- 10  **containerd**
Pulls the image and creates the container(s) using OCI runtime.
- 11  **Containers Running**
Your application containers are now running.

Who Does What?

- You (kubectl)**
Sends the desired state to the cluster.
- API Server**
Validates the request and stores it in etcd.
- etcd**
Stores the cluster state (desired state).
- Deployment Controller**
Observes the Deployments and creates a ReplicaSet to match the desired replicas.
- ReplicaSet Controller**
Creates Pod objects to match the desired number of replicas.
- API Server / etcd**
Stores the new Pod objects (in Pending state).
- Scheduler**
Finds a suitable node and updates the Pod with the selected node.
- API Server / etcd**
Stores the binding information (nodeName).
- kubelet (on the node)**
Pulls images, creates containers and manages the Pod lifecycle.
- containerd (CRI)**
Downloads the image and runs the container via runc (OCI runtime).
- Your Application**
Containers are running and the Pod is Ready.

Status Flows Back Up

-  **Pod status (Running)**
Reported by kubelet
-  **Node status**
Updated by kubelet
-  **Stored in etcd**
Latest state persisted
-  **API Server**
Serves updated status to clients (kubectl, dashboards, etc.)
-  **kubectl get pods**
You see the current status

Asynchronous Architecture (Senior Insight)

- There is no single central process performing all the steps.
- Each component watches the cluster state and reacts independently.
- Controllers, scheduler and kubelet run reconciliation loops.
- The system is eventually consistent.
- This design makes Kubernetes highly scalable, resilient and self-healing.

Real-World Analogy



You place an order (Deployment). Different teams work on it (controllers, scheduler, kubelet, runtime). You don't watch them directly, but you get the final product (running containers).

Key Takeaways

- ✓ A Deployment creates a ReplicaSet, which creates Pods.
- ✓ The scheduler chooses the node.
- ✓ kubelet and container runtime make the containers run.
- ✓ Status flows back up to the API server and to you.
- ✓ Kubernetes is event-driven and asynchronous.
- ✓ Many independent components work together.



Junior Engineer Tip

Use **kubectl describe** at each stage (Deployment, ReplicaSet, Pod, Node) to see what is happening. It helps you understand where things are stuck.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KUBERNETES NETWORKING

Four Networks to Understand

Kubernetes networking solves four main questions. Understand these networks to design, troubleshoot and expose your applications correctly.

1 Inside the Pod

How does a container communicate inside its Pod?

2 Pod to Pod

How do Pods communicate with other Pods?

3 Finding Services

How do applications find Services?

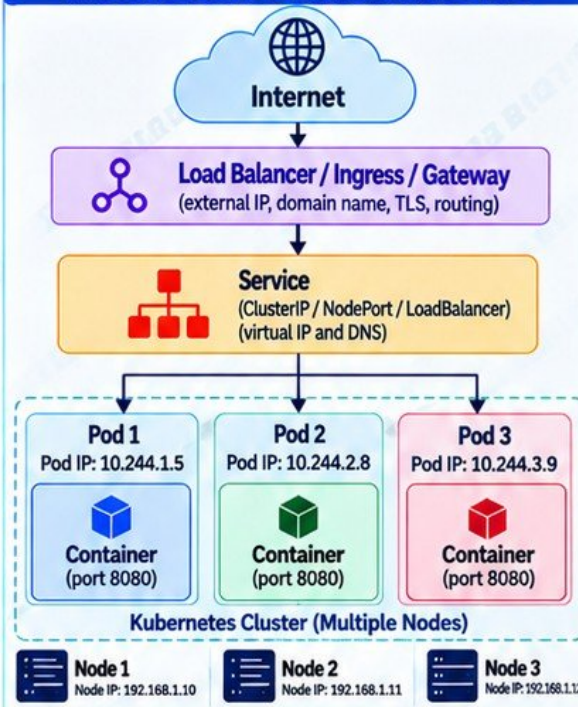
4 External Access

How does external traffic enter the cluster?

The Networking Flow

- Internet**
External clients (users, other systems).
- Load Balancer / Ingress / Gateway**
Brings external traffic into the cluster.
- Service**
Stable virtual IP and DNS name that load balances to Pods.
- Pod IP**
Each Pod has its own IP address (reachable within the cluster).
- Container**
Your application running inside the container.

Kubernetes Network Architecture



Understanding the Addresses

- Node IP**
 - IP address of the worker node
 - Used for node-to-node communication
 - Used by NodePort (external access)
- Pod IP**
 - Unique IP for each Pod
 - Routable within the cluster
 - Pods can communicate directly (flat network model)
- Service IP**
 - Virtual IP (ClusterIP)
 - Stable, does not change
 - Used for service discovery
 - Load balances to Pod IPs
- External IP**
 - Public IP (from cloud or on-prem)
 - Used by LoadBalancer, Ingress or Gateway
 - Allows traffic from the internet

Junior Engineer Mistakes

- ✗ Confusing Node IP, Pod IP, Service IP and External IP.
- ✗ Trying to access a Pod directly from the internet.
- ✗ Thinking a Service is a Pod.
- ✗ Using the wrong port or IP.
- ✗ Not understanding how DNS works in the cluster.

Key Networking Concepts (Senior Level)

- Flat Pod Networking Model**
All Pods can communicate with each other (no NAT).
- Overlays**
Tools like Flannel, Calico or Cilium create an overlay network (e.g. VXLAN, IPIP, WireGuard).
- Routing**
Ensures traffic is delivered between nodes and Pods.
- Encapsulation**
Packets are wrapped (e.g. VXLAN) and sent across nodes.
- eBPF Dataplanes**
Modern CNIs (e.g. Cilium) use eBPF for high performance networking, security and observability.

Service Types Recap

- ClusterIP** (default, internal only)
- NodePort** (exposes on each node IP)
- LoadBalancer** (provisions external IP)
- Ingress / Gateway** (HTTP/HTTPS routing, domain names)

Important Notes

- Every Pod has an IP. Every Service has a virtual IP.
- Pods can talk to each other directly.
- Services provide stable access to Pods.
- Ingress/Gateway handle external HTTP/HTTPS traffic.
- Network policies can control traffic between Pods.
- Use DNS names instead of hardcoding IPs.

Key Takeaways

- ✓ Understand the four networking questions.
- ✓ Know the difference between Node IP, Pod IP, Service IP and External IP.
- ✓ Services provide stable access to applications.
- ✓ Kubernetes uses a flat Pod networking model.
- ✓ CNI plugins (e.g. Calico, Cilium, Flannel) provide the networking.
- ✓ eBPF is becoming the standard for high performance networking.
- ✓ Correct networking knowledge is essential for secure and reliable applications.

Junior Engineer Tip

Use `kubectl get pods -o wide` and `kubectl get svc` to see the real IPs in your cluster. This helps you understand how the networking actually works.

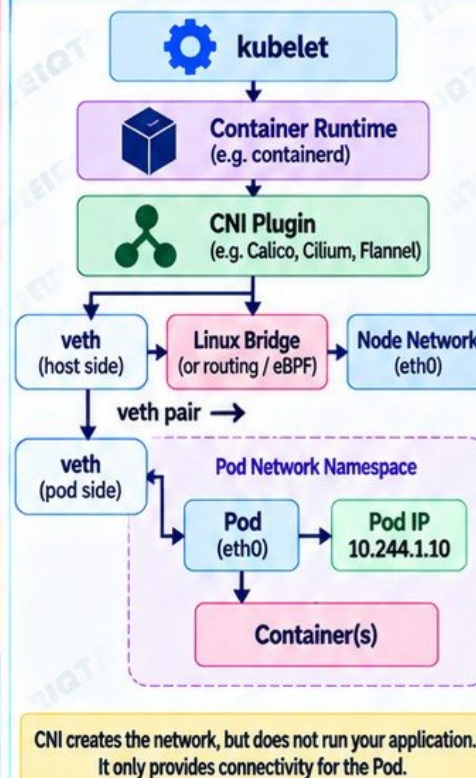
CNI: How Pods Get Network Connectivity

The Container Network Interface (CNI) is a plugin system that configures networking for Pods. It is responsible for creating network interfaces, assigning IP addresses and setting up the necessary routes so that Pods can communicate.

The CNI Flow (Step by Step)

- kubelet**
Kubelet asks the container runtime to create a Pod sandbox.
- Runtime creates Pod sandbox**
Container runtime (e.g. containerd) creates the network namespace for the Pod.
- CNI plugin**
Runtime calls the CNI plugin (e.g. Calico, Cilium, Flannel).
- Network interface created**
CNI creates a network interface (veth pair) and attaches it to the node network.
- Pod receives IP**
CNI assigns an IP address to the Pod using IPAM.
- Routes configured**
CNI sets up the necessary routes, iptables/eBPF rules and policies for connectivity.

CNI Architecture on a Node



Common CNI Implementations

- Cilium**
 - eBPF based
 - High performance
 - Advanced networking and security
- Calico**
 - BGP or overlays
 - Widely used
 - Network policy support
- Flannel**
 - Simple overlay network (VXLAN)
 - Easy to set up
 - Common in small clusters
- Cloud-Specific CNIs**
 - AWS VPC CNI (EKS)
 - Azure CNI (AKS)
 - GCP VPC CNI (GKE)
 - Use native cloud networking

Key Concepts

- veth pairs**
Virtual Ethernet pairs connecting the Pod and the host network.
- Bridges**
Linux bridge connects veth to the node.
- Routing**
Routes ensure traffic reaches other Pods and networks.
- Overlays**
Encapsulation (e.g. VXLAN) used to connect Pods across nodes (at a conceptual level).

Networking Technologies (Senior Level)

- VXLAN**
Encapsulates Pod traffic over an underlay network.
- BGP**
Used for routing Pod networks between nodes (with Calico or Cilium).
- eBPF Dataplanes**
Modern CNIs (e.g. Cilium) use eBPF for high performance, security and observability.
- Native Routing**
Some CNIs use native routing without overlays.
- IPAM (IP Address Management)**
Allocates and manages IP addresses for Pods.

Troubleshooting CNI

- Check Pod IPs:**
`kubectl get pods -o wide`
- Check network interfaces on the node:**
`ip addr`
- Check routes:**
`ip route`
- Check CNI logs (example for containerd):**
`journalctl -u containerd -f`

Common Issues

- Pod gets no IP address
- Pods cannot communicate across nodes
- CNI plugin not installed or misconfigured
- IP address conflicts (IPAM issues)
- Network policies blocking traffic
- DNS not working
- Wrong routes or MTU issues
- Overlay network problems

Junior Engineer Tip

When a Pod has no network connectivity, first check if the CNI plugin is running correctly and look at the Pod IP, node interfaces and routes.

Services, kube-proxy and the Data Plane

Pod IPs are not stable. Pods are created, deleted and replaced. Services provide a stable access point, and the data plane (kube-proxy or eBPF) forwards traffic to the right Pods.

Why Pod IPs Alone Are Insufficient

- ✗ Pods are ephemeral (they can be deleted and recreated).
- ✗ Pod IPs change frequently.
- ✗ It is hard for clients to keep track of Pod IPs.
- ✗ Services provide a stable, virtual IP and DNS name that always points to the right Pods.

Key Concepts



Service

- Stable virtual IP and DNS name.
- Load balances traffic to healthy Pods.



ClusterIP

- Internal virtual IP (only inside cluster).
- Accessible via DNS (e.g. myapp.default.svc).



EndpointSlice

- Tracks the actual Pod IPs behind a Service (more scalable than Endpoints).
- Updated automatically as Pods change.



kube-proxy

- Runs on each node.
- Implements the Service dataplane (iptables, IPVS or eBPF).



iptables

- Older mode for packet forwarding.
- Uses Linux iptables rules.



IPVS

- Higher performance than iptables.
- Uses Linux IPVS for load balancing.



eBPF Alternatives

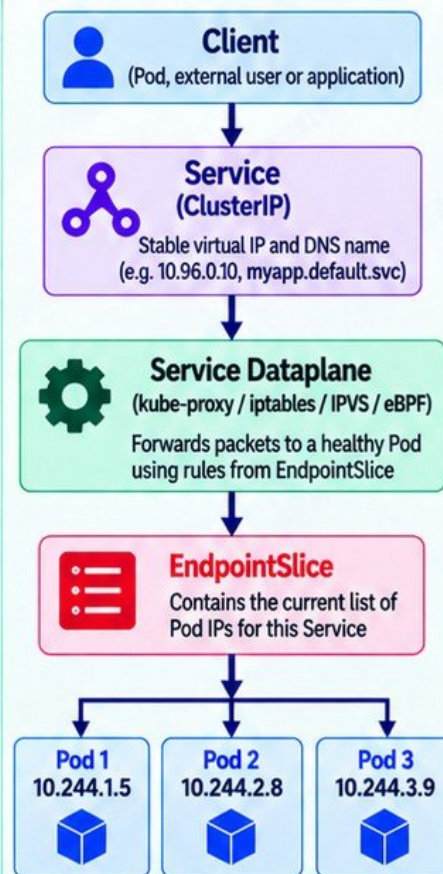
- Modern, high-performance dataplanes (e.g. Cilium).
- Can replace kube-proxy entirely.

Useful Commands



```
# List services
kubectl get svc
# Check endpoint slices
kubectl get endpointslices
# Describe a service
kubectl describe svc <service-name>
# See pods and their IPs
kubectl get pods -o wide
```

Request Flow (Service to Pod)



How kube-proxy Works

- Watches Services and EndpointSlices from the API server.
- Programs the node's dataplane (iptables, IPVS or eBPF).
- Ensures traffic to the Service ClusterIP is forwarded to one of the healthy Pods.

Control Plane vs Data Plane



Control Plane (Decides State)

- API server stores Services and EndpointSlices.
- Controllers ensure they match the desired state.
- Decides what should exist.



Data Plane (Forwards Packets)

- kube-proxy, iptables, IPVS or eBPF forward actual network traffic.
- Runs on each node.
- Operates in real time.
- Forwards packets to the correct Pod IPs.

Common Failure



Service exists but has no endpoints.

Usually caused by:

- No matching Pod labels.
- Pods not ready.
- Selector misconfiguration.
- Pod crash or not running.

Service Types (Quick Recap)

ClusterIP	→ Internal access only (default)
NodePort	→ Exposes service on every node IP
LoadBalancer	→ Provision external load balancer
ExternalName	→ Maps to external DNS name

Key Takeaways

- ✓ Pod IPs change. Use Services for stable access.
- ✓ Services use ClusterIP and DNS names.
- ✓ EndpointSlice tracks the actual Pod IPs.
- ✓ kube-proxy (iptables, IPVS or eBPF) forwards traffic to the right Pod.
- ✓ Control plane decides state, data plane forwards packets.
- ✓ If a Service has no endpoints, check labels, Pod status and selectors.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

DNS and Service Discovery

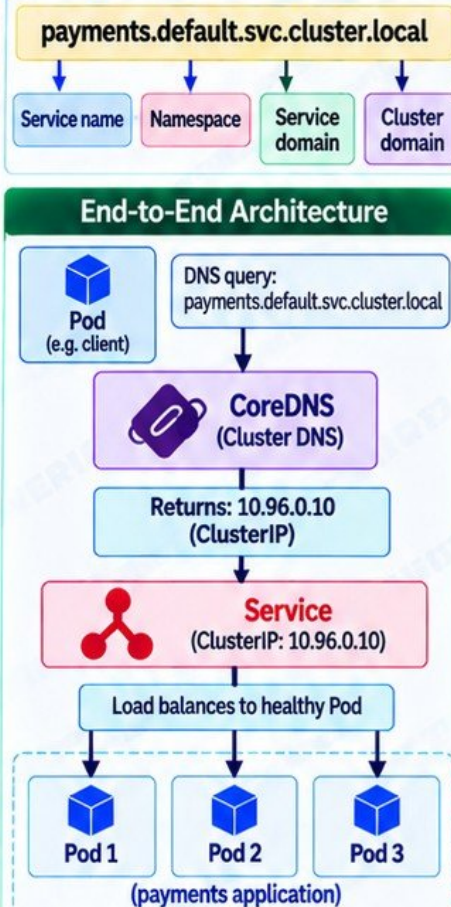
How Kubernetes Finds Your Application

Kubernetes uses DNS (provided by CoreDNS) to give Services stable, human-readable names that resolve to ClusterIP addresses, allowing Pods to find and communicate with applications.

The DNS Flow (Step by Step)

- Pod**
An application in a Pod makes a request to a service name (e.g. payments.default.svc.cluster.local).
- DNS Query**
The Pod's DNS resolver sends a DNS query to CoreDNS.
- CoreDNS**
CoreDNS receives the query and looks up the Service record in the cluster.
- Service DNS Record**
CoreDNS returns the ClusterIP for the Service.
- ClusterIP**
The Pod receives the ClusterIP (e.g. 10.96.0.10).
- Service**
Traffic is sent to the Service (which load balances to a healthy Pod).
- Pod**
The request reaches one of the backend Pods running the application.

Example: How a Service Name Resolves



Key Concepts

- CoreDNS**
Default DNS server in Kubernetes. Resolves Service names to ClusterIPs.
- Service Names**
Format: <service>.<namespace>.svc.cluster.local
Example: payments.default.svc.cluster.local
- Namespaces**
Services are namespaced. Use the correct namespace.
- Search Domains**
Kubernetes automatically adds search domains to simplify DNS queries.
- FQDN**
Fully Qualified Domain Name. Example: payments.default.svc.cluster.local
- /etc/resolv.conf**
Contains the DNS server (CoreDNS IP) and search domains inside the Pod.

Example /etc/resolv.conf

```
nameserver 10.96.0.10
search default.svc.cluster.local
svc.cluster.local cluster.local
options ndots:5
```

Useful Commands (Troubleshooting)

```
> # Test DNS resolution
nslookup kubernetes.default
# or using dig
dig kubernetes.default

# Check CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns

# Check CoreDNS logs
kubectl logs -n kube-system -l k8s-app=kube-dns
```

Senior-Level Topics

- ndots**
Controls when a query is treated as FQDN vs relative.
- DNS Query Amplification**
Short search domains can generate multiple queries.
- CoreDNS Caching**
Reduces latency and load on the DNS server.
- NodeLocal DNSCache**
Improves performance by running a local DNS cache on each node.
- DNS Scaling**
Configure replicas, resources and caching for large clusters.

Common Issues

- ✗ DNS query fails (no such host)
- ✗ Service exists but returns the wrong IP
- ✗ Pod cannot resolve Service name
- ✗ CoreDNS pod is not running
- ✗ High DNS latency
- ✗ Misconfigured search domains
- ✗ Network policies blocking DNS (port 53)
- ✗ Using the wrong namespace

Key Takeaways

- ✓ Kubernetes provides built-in DNS via CoreDNS.
- ✓ Use Service names instead of Pod IPs.
- ✓ Understand namespaces and FQDN format.
- ✓ Check /etc/resolv.conf inside Pods.
- ✓ Use nslookup or dig for troubleshooting.
- ✓ Tune ndots and use NodeLocal DNSCache in large clusters.

Junior Engineer Tip

If a service is not reachable, first check DNS resolution. If the name does not resolve, the issue is DNS. If it resolves but still fails, check the Service, endpoints and network policies.

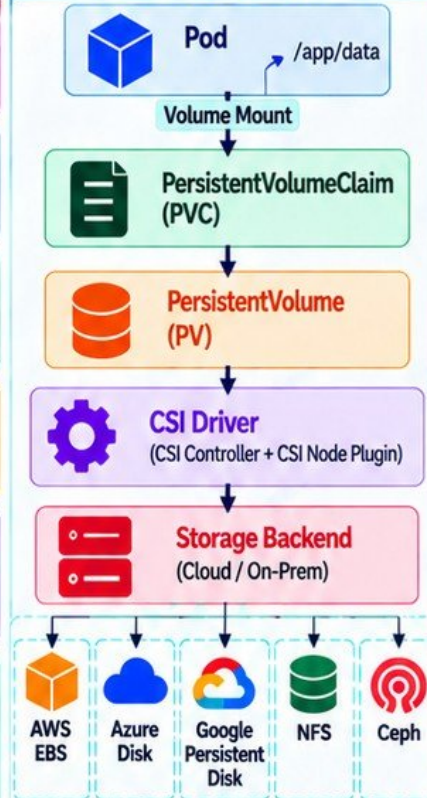
Kubernetes Storage: From Pod to Physical Disk

Kubernetes uses an abstraction layer to manage storage. A Pod does not talk directly to a cloud disk. It uses PVCs, PVs and CSI drivers to provision, attach and mount storage from the underlying infrastructure.

The Storage Chain (Step by Step)

- Pod**
A Pod requests storage and mounts it as a volume.
- Volume Mount**
The Pod mounts the volume inside the container.
- PersistentVolumeClaim (PVC)**
The Pod uses a PVC to request storage (size, access mode, etc).
- PersistentVolume (PV)**
A PV represents a real piece of storage in the cluster.
- CSI Driver**
The Container Storage Interface (CSI) driver provisions, attaches and mounts the volume.
- Storage Backend**
The actual storage in your infrastructure (cloud or on-prem).

Architecture Diagram



Key Components Explained

- PersistentVolumeClaim (PVC)**
 - User request for storage.
 - Defines size, access mode, StorageClass, etc.
- PersistentVolume (PV)**
 - Actual storage resource in the cluster.
 - Can be dynamically or statically provisioned.
- StorageClass**
 - Defines how storage should be provisioned.
 - Specifies the CSI driver, parameters, etc.
- CSI Driver**
 - Communicates with the storage backend.
 - Consists of a controller plugin and a node plugin.

Access Modes (Common)

- ReadWriteOnce (RWO)** – mounted on one node
- ReadOnlyMany (ROX)** – read-only, many nodes
- ReadWriteMany (RWX)** – read-write, many nodes

Important Concepts

- Dynamic provisioning** – PVs created automatically using a StorageClass.
- Static provisioning** – PVs created manually by an admin.
- Reclaim policy** – what happens when a PVC is deleted (Retain, Delete, Recycle (deprecated)).
- Volume expansion** – increase the size of a PVC.
- Topology** – storage can be limited to specific zones or regions.

CSI Components (Senior Level)

- CSI Controller**
 - Handles provisioning, attachment and deletion of volumes (runs in the control plane).
- CSI Node Plugin**
 - Runs on each node.
 - Handles volume mounting and unmounting.
- Attachment**
 - Attaches the volume to the node (for block storage).
- Mounting**
 - Formats (if needed) and mounts the filesystem inside the node, then into the Pod.
- Topology**
 - Supports zone/region aware provisioning.

Common Failure: ContainerCreating

❌ A Pod can get stuck in ContainerCreating if the volume cannot be attached or mounted.

Common causes:

- Wrong StorageClass.
- Insufficient permissions (IAM).
- Volume not found.
- Attachment failure (wrong AZ/zone).
- Node plugin not running.
- Mount errors (filesystem issues).
- Network or API issues with the storage backend.
- PV/PVC mismatch.

Useful Commands (Troubleshooting)

```

# Check PVC
kubectl get pvc
kubectl describe pvc <pvc-name>

# Check PV
kubectl get pv
kubectl describe pv <pv-name>

# Check StorageClass
kubectl get storageclass

# Check events
kubectl get events --sort-by=.metadata.creationTimestamp

# Describe the Pod
kubectl describe pod <pod-name>
  
```

Junior Engineer Tip

When a Pod is stuck in ContainerCreating, always check the PVC, PV, events and the storage backend logs. Most storage issues are not application issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Kubernetes Identity and Security Path

Kubernetes secures access using authentication, authorization, RBAC and admission control. Every request to the API server goes through these layers before a resource is accepted or rejected.

Key Identities in Kubernetes

- Users**
 - Humans or systems that access the cluster (e.g. via kubectl).
 - Authenticated using certificates, OIDC, or external identity providers.
- ServiceAccounts**
 - Identities for applications running inside Pods.
 - Used by Pods to access the API server.
- Secrets**
 - Store sensitive data (tokens, passwords, keys).
 - Often used with ServiceAccounts.

RBAC Building Blocks

- Role**
 - Defines permissions within a single namespace.
- ClusterRole**
 - Defines permissions across the entire cluster.
- RoleBinding**
 - Binds a Role to a user, group or ServiceAccount (namespace).
- ClusterRoleBinding**
 - Binds a ClusterRole to a user, group or ServiceAccount (cluster-wide).

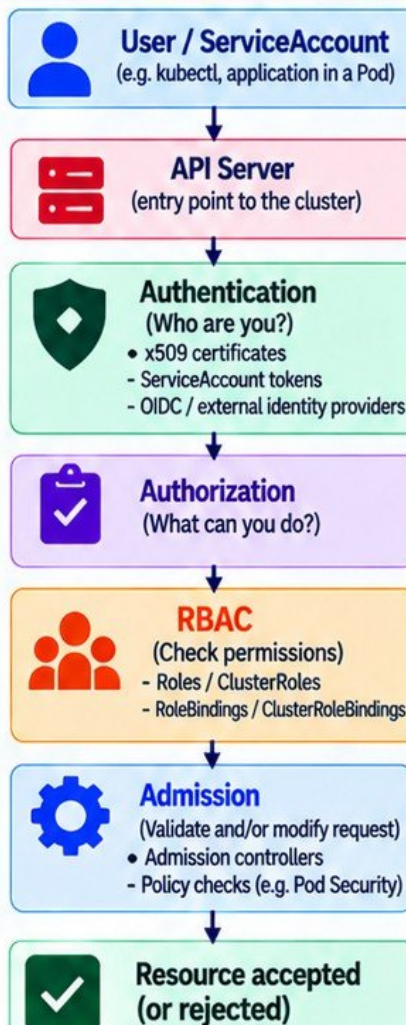
Least Privilege

- Give only the permissions that are needed.
- Use specific Roles instead of cluster-admin.
- Regularly review and audit permissions.
- Avoid long-lived tokens and credentials.
- Use namespaces to isolate access.

Useful Commands

```
# Check current user
kubectl auth whoami
# Check permissions
kubectl auth can-i <verb> <resource>
# View roles and bindings
kubectl get roles,rolebindings
kubectl get clusterroles,clusterrolebindings
```

The Secured Request Flow



Important Components

- Admission Controllers**
 - Validate or modify requests to the API server.
 - Examples: Pod Security Admission, MutatingWebhook, ValidatingWebhook.
- Pod Security Admission (PSA)**
 - Enforces security standards for Pods (restricted, baseline, privileged).
 - Replaces the older PodSecurityPolicy.
- Secrets**
 - Store tokens, passwords, certificates, etc.
 - Use with care. Consider external secrets solutions for production.

Senior-Level Topics

- OIDC (OpenID Connect)**
 - Integrate Kubernetes with external identity providers (e.g. Microsoft Entra ID, Okta).
 - Enables SSO for users.
- Certificates**
 - Used for secure communication and user authentication (x509).
 - Managed by kube-apiserver and controllers.
- Authorization Modes**
 - RBAC (default), Node, ABAC, Webhook.
 - Multiple modes can be enabled.
- Admission Webhooks**
 - Custom validation or mutation logic.
 - Used for policy enforcement.
- API Audit Logs**
 - Record all requests to the API server.
 - Useful for security monitoring and compliance.

Security Reminder

- Never treat cluster-admin as the default solution to RBAC problems.**
- Use least privilege, understand the permissions you need, and assign only what is required.

Common Mistakes

- ✗ Giving cluster-admin to everyone.
- ✗ Using overly broad permissions.
- ✗ Not using namespaces.
- ✗ Exposing secrets in plain text.
- ✗ Not auditing access.

High Availability and Failure Domains

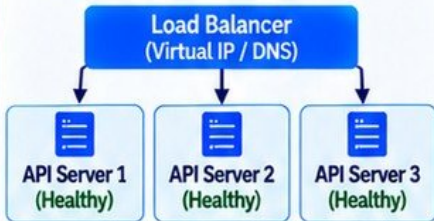
Kubernetes is designed to be highly available. Multiple control plane nodes, etcd members and worker nodes ensure that the cluster continues to run even when individual components fail.

Why High Availability Matters

- ✓ Avoids single points of failure.
- ✓ Keeps applications running.
- ✓ Handles node, zone or component failures automatically.
- ✓ Provides a better user experience and reliability.
- ✓ Essential for production environments.

Control Plane High Availability

- Run multiple API servers.
- Use a load balancer in front of them.
- Keep etcd in a separate, redundant cluster.
- If one control-plane node fails, others continue to serve requests.



etcd High Availability

- Run an odd number of etcd members (3, 5, or 7).
- Requires a quorum (majority) to make changes.
- Stores the entire cluster state.
- If quorum is lost, the cluster cannot make changes (read-only or unavailable).



High Availability vs Backup



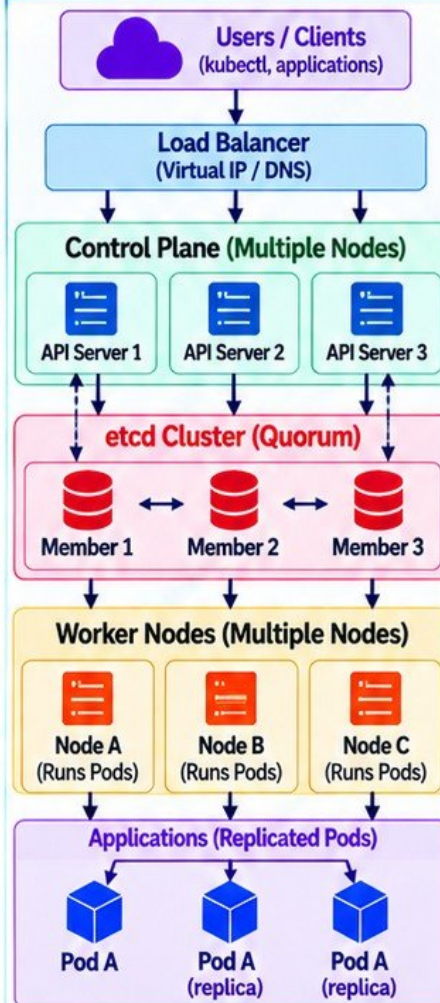
High Availability

- Prevents downtime.
- Uses redundancy.
- Handles failures automatically.
- Keeps services running.

Backup

- Used for disaster recovery.
- Restores data after major loss.
- Does not prevent downtime.
- Needed even with high availability.

Kubernetes HA Architecture



What Happens When Components Fail?

- ✗ One API server fails → others continue serving.
- ✗ One etcd member fails → cluster still works (if quorum).
- ✗ One worker node fails → Pods are rescheduled.
- ✗ One zone fails → workloads in other zones keep running (if properly distributed).
- ✗ If etcd loses quorum → cluster cannot make changes.
- ✗ If all control-plane nodes fail → cluster is unavailable until restored.

Worker Node High Availability

- Use multiple worker nodes.
- Distribute Pods across nodes and zones.
- If a node fails, Pods are rescheduled to healthy nodes.
- Use node labels, taints and tolerations when needed.
- Combine with PodDisruptionBudgets (PDBs) for planned disruptions.



Pods distributed across nodes and zones (using replicas, anti-affinity, topology spread)

Key Concepts



Quorum

Majority of etcd members must be available to make changes (e.g. 2 out of 3).



Replica Distribution

Run multiple replicas and spread them across nodes and zones.



Availability Zones

Use multiple zones to protect against datacenter failure.



Anti-Affinity

Helps ensure Pods are not scheduled on the same node or zone.



PodDisruptionBudgets (PDBs)

Limits the number of Pods that can be unavailable during maintenance.



Senior-Level Topics

- Control-plane quorum and leader election.
- Zone failure and regional resilience.
- Topology spread constraints.
- Failure domains (node, rack, zone, region).
- Split brain concepts and how to prevent them.
- Designing for multi-region clusters (advanced).



Key Takeaways

- Use multiple control-plane nodes, etcd members and worker nodes.
- Distribute workloads across zones.
- Use anti-affinity, topology spread and PDBs.
- High availability is not the same as backup.
- Plan for failure, not just recovery.
- Test your failure scenarios.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQTA

ENGINEER A BETTER TOMORROW

Knowledge
Builds Freedom

How to Troubleshoot a Kubernetes Cluster Systematically

A structured approach helps you find and fix problems faster. Start where the symptom appears, then walk downward through dependencies. Do not randomly restart components.

Identify
Investigate
Solve
Learn
Repeat

Troubleshooting Ladder (Check from top to bottom)

- 1  **Application**
Is the application working?
Check logs, errors and behaviour.
- 2  **Pod**
Is the Pod running?
Check status, events and logs.
- 3  **Container**
Is the container running?
Check logs, processes and health.
- 4  **Node**
Is the node healthy?
Check resources, condition and logs.
- 5  **Service**
Is the Service working?
Check endpoints and connectivity.
- 6  **Network**
Can components communicate?
Check DNS, routes, policies.
- 7  **Storage**
Is the storage working?
Check PVC, PV and backend.
- 8  **Control Plane**
Are control plane components healthy?
Check API server, etcd, controller manager, scheduler.

Troubleshooting Workflow

- 1 **Observe symptom**
Understand what is not working.
- 2 **Check Kubernetes state**
Use kubectl to get an overview.
- 3 **Inspect Events**
Look for error messages and warnings.
- 4 **Inspect logs**
Check Pod, container and component logs.
- 5 **Check node**
Verify node health and resources.
- 6 **Check dependencies**
Look at Services, Network, Storage, etc.
- 7 **Form hypothesis**
Identify the most likely cause.
- 8 **Test hypothesis**
Run commands or make a small change.
- 9 **Fix**
Apply the correct solution.
- 10 **Verify**
Confirm the issue is resolved.

Core Kubernetes Commands

```
>... # Get resources
kubectl get pods
kubectl get svc
kubectl get nodes

# Describe resource (detailed info)
kubectl describe pod <pod-name>

# View logs
kubectl logs <pod-name>
kubectl logs -f <pod-name>

# View events
kubectl get events --sort-by=.metadata.creationTimestamp

# Check resource usage
kubectl top pods
kubectl top nodes
```

Additional Useful Tools (Deeper Debugging)

```
# Node logs
journalctl -u kubelet -f

# Container runtime
crictl ps
crictl logs <container-id>

# Network troubleshooting
ss -tulnp
ip addr
ip route
dig <service-name>
curl -v <service-ip>:<port>
```

Common Issues and What to Check

- ✗ Pod stuck in Pending → Check node, resources, scheduling.
- ✗ Pod stuck in ContainerCreating → Check storage, CNI, events.
- ✗ Pod keeps restarting → Check logs, probes, resource limits.
- ✗ Service has no endpoints → Check selectors and Pod labels.
- ✗ Cannot reach service → Check DNS, network policies, firewall.
- ✗ Slow application → Check resources, logs, dependencies.
- ✗ Node NotReady → Check node logs, network, resources.

"Don't guess. Observe. Investigate.
Understand. Then fix."



A healthy cluster is not a lucky cluster.
It is a well-monitored, well-understood
and well-maintained cluster.

Senior-Level Tips

- ✓ Start where the symptom appears, then walk downward through dependencies.
- ✓ Use events and logs before making changes.
- ✓ Understand the relationships between components.
- ✓ Do not randomly restart components.
- ✓ Check for patterns and recent changes.
- ✓ Use tools like crictl and journalctl on nodes.
- ✓ Document the root cause and solution.

Key Takeaways

- ✓ Use a systematic approach.
- ✓ Kubernetes provides powerful tools for troubleshooting.
- ✓ Understand how components depend on each other.
- ✓ Check events, logs and resource status.
- ✓ Test your hypothesis before applying a fix.
- ✓ Verification is important.
- ✓ Troubleshooting gets easier with practice.



Problems are not setbacks.
They are training for real engineering.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

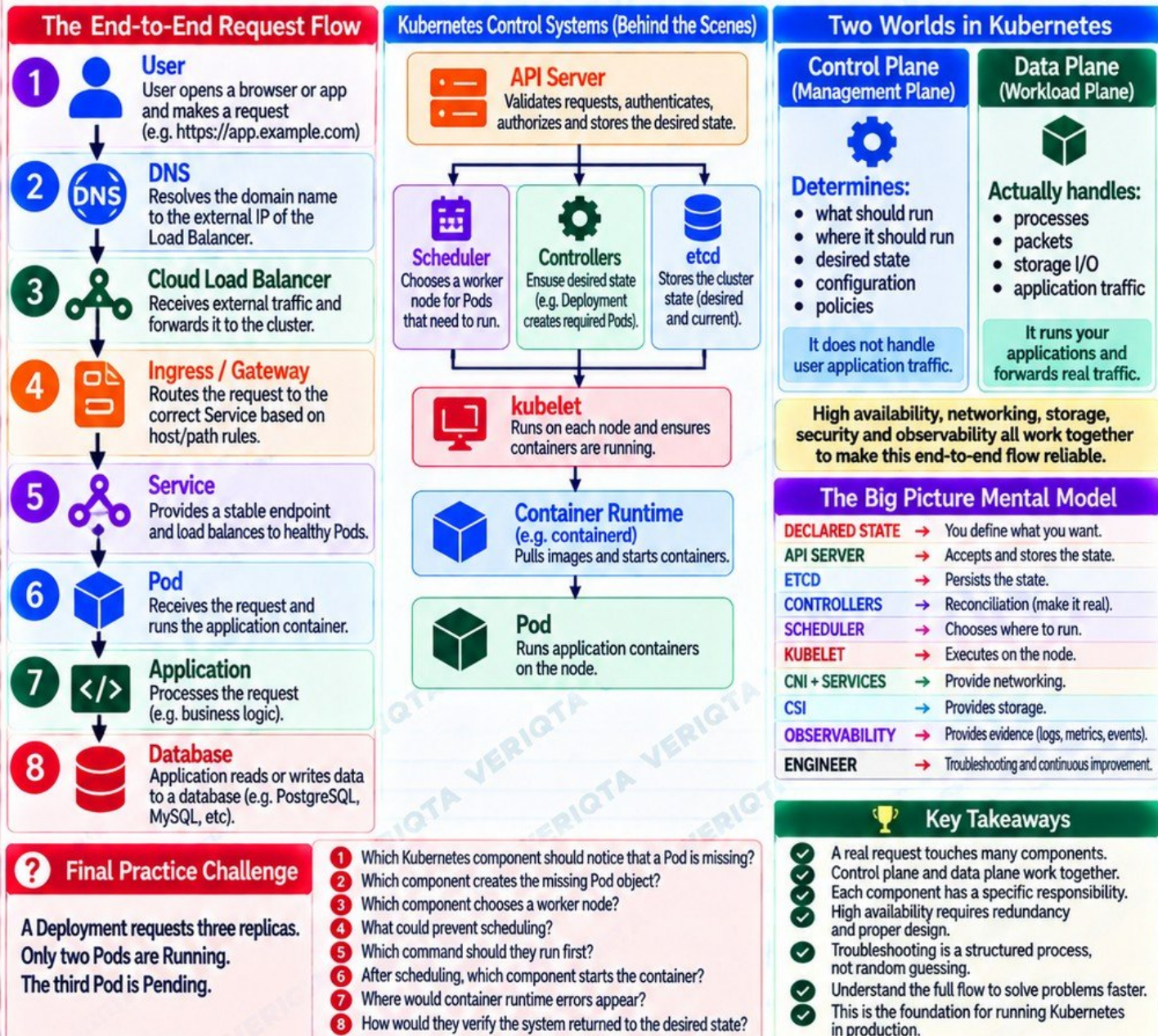
VERIQTA

ENGINEER A BETTER TOMORROW

Knowledge
Builds Freedom

One Request Through the Entire Kubernetes Cluster

A real user request involves many components working together.
This page ties everything together and shows the complete flow.



"The more you understand the whole system, the easier it becomes to solve any problem."

You are now ready to think like a Kubernetes engineer.

Learn. Practice. Build. Grow.
A better you. A better tomorrow.